

Final Database Design

1. Provider Management Service

Database

provider_db

Table: service_categories

Column	Type	Constraints
category_id	INT	PK, AUTO_INCREMENT
category_name	VARCHAR(50)	NOT NULL, UNIQUE
description	VARCHAR(255)	NULL

Table: providers

Column	Type	Constraints
provider_id	INT	PK, AUTO_INCREMENT
category_id	INT	FK
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL
Column	Type	Constraints
phone	VARCHAR(15)	UNIQUE
location	VARCHAR(100)	NOT NULL
experience_years	INT	DEFAULT 0
availability	ENUM('Available','Busy','Offline')	DEFAULT 'Available'
rating	DECIMAL(2,1)	DEFAULT 0.0
profile_image	VARCHAR(255)	NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Relationship

Service Category (1)



Provider (Many)

2. Booking Management Service

Database

booking_db

Table: customers

Column	Type	Constraints
customer_id	INT	PK, AUTO_INCREMENT
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
email	VARCHAR(100)	UNIQUE
password	VARCHAR(255)	NOT NULL
phone	VARCHAR(15)	UNIQUE
address	VARCHAR(255)	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Table: bookings

Column	Type	Constraints
booking_id	INT	PK, AUTO_INCREMENT
customer_id	INT	FK
provider_id	INT	API Reference
booking_date	DATE	NOT NULL
booking_time	TIME	NOT NULL
service_address	VARCHAR(255)	NOT NULL
description	TEXT	NULL
status	ENUM('Pending','Accepted','Rejected','In Progress','Completed','Cancelled')	DEFAULT 'Pending'
payment_status	ENUM('Pending','Paid','Failed')	DEFAULT 'Pending'
total_amount	DECIMAL(10,2)	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Table: payments

Column	Type	Constraints
payment_id	INT	PK, AUTO_INCREMENT
booking_id	INT	FK
payment_method	VARCHAR(30)	NOT NULL
amount	DECIMAL(10,2)	NOT NULL
payment_status	ENUM('Success','Pending','Failed')	DEFAULT 'Pending'
transaction_reference	VARCHAR(100)	UNIQUE
payment_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Relationships

Customer (1)

|

▼

Bookings (Many)

|

▼

Payments (1)

3. Notification Service

Database

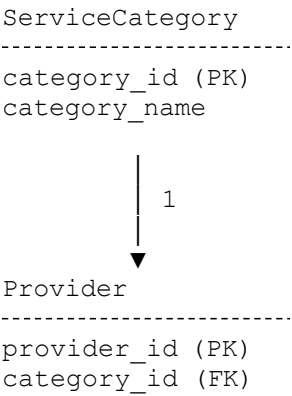
notification_db

Table: notifications

Column		Type	Constraints
notification_id	INT		PK, AUTO_INCREMENT
booking_id	INT		API Reference
user_id	INT		API Reference
user_type	ENUM('Customer','Provider')		NOT NULL
title	VARCHAR(100)		NOT NULL
message	TEXT		NOT NULL
status	ENUM('Unread','Read')		DEFAULT 'Unread'
notification_type	ENUM('Booking','Payment','Reminder')		NOT NULL
created_at	TIMESTAMP		DEFAULT CURRENT_TIMESTAMP

Full Relationship

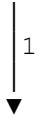
Provider Database



Booking Database

Customer

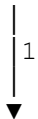
customer_id (PK)



Booking

booking_id (PK)

customer_id (FK)



Payment

payment_id (PK)

booking_id (FK)

Notification Database

Notification

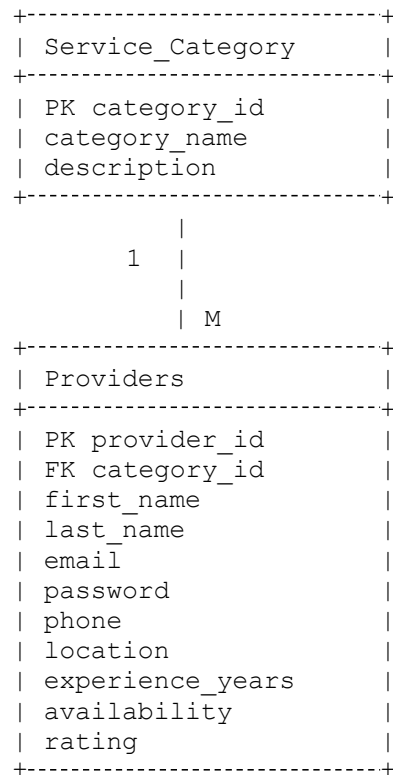
notification_id (PK)

booking_id

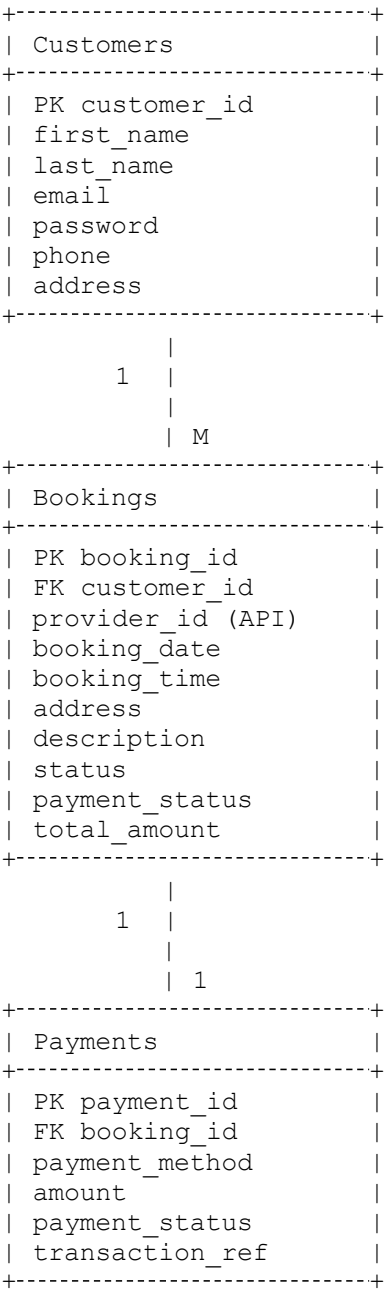
user_id

Notice that `provider_id`, `booking_id`, and `user_id` in different databases are **API references**, not database foreign keys. This is a key SOA principle: each microservice owns its own database, and services communicate through REST APIs rather than cross-database joins.

Complete EER Diagram



=====



Notifications
PK notification_id
booking_id (API)
user_id (API)
user_type
title
message
status
notification_type

Important SOA Note

Add to report:

- **Inside the same database, use foreign keys:**
 - service_categories → providers
 - customers → bookings
 - bookings → payments
- **Between different microservices, do not use foreign keys. Instead, use REST API communication.** For example:
 - Booking Service calls Provider Service to get provider details.
 - Notification Service receives booking events and creates notifications.
 - Payment Gateway communicates with Booking Service to update payment status.